# Asistente RAG reproducible sobre prensa y Bluesky con Milvus y modelos locales: arquitectura, evaluación y compuerta de dominio

Erika Valeria Rivera Jimenez, Julian Guillermo Carrillo Meneses,
Carlos Andres Garnica Salazar, Oscar Andres Prieto Moreno

Pontificia Universidad Javeriana, Bogotá, Colombia
riveraev@javeriana.edu.co, carrillojuliang@javeriana.edu.co,
carlos_garnicas@javeriana.edu.co, o-prieto@javeriana.edu.co

***Resumen*: La recuperación aumentada por generación (RAG) permite responder sobre un corpus sin reentrenar el modelo, pero su calidad depende de decisiones que rara vez se documentan juntas: limpieza, segmentación, codificador, índice, reordenamiento, prompt y control de abstención. Este trabajo presenta un asistente RAG reproducible sobre 169.248 documentos recolectados de prensa colombiana e internacional y Bluesky, 161.868 indexados tras descartar páginas de error y 618.286 fragmentos en Milvus, con modelos locales. La comparación de capas es exploratoria sobre el silver completo. Se comparan las capas con experimentos pareados donde existen intervalos; las demás comparaciones son estimaciones puntuales exploratorias: la limpieza conservadora no mejora de forma significativa, el chunking fijo tiende a superar a la segmentación por oraciones y es más rápido, bge-m3 obtiene mejor recall que e5-base en la estimación observada, y la unión con BM25 más el reordenamiento en GPU sube el recall@10 del conjunto de desarrollo de 0,600 a 0,700. En el silver exploratorio el sistema entrega 37 de 38 preguntas respondibles, se abstiene en las 5 sin respuesta y cubre los tokens de referencia en 25 de 37. En un conjunto reservado de 20 preguntas nuevas, entrega 19, cubre los tokens de referencia en 17 y se abstiene en las 5 sin respuesta; el recall@10 es 0,700 y la compuerta rechaza las 5 sin respuesta y 3 de las 20 directas. En 15 preguntas únicas y 42 con doble anotación, el detector alcanza F1 de 1,0 en las primeras y 0,894 en las 42; el resultado automático del condicional es 33 de 40 (0,825) frente a 30 de 40 (0,750) del baseline, con McNemar p=0,375; la cobertura entre opciones es 18 de 21; la clarificación no se separa (29 de 40, p=1,0). Sobre el silver asistido por LLM sube a 36 de 42 (p=0,039), con 22 de 25 de cobertura entre opciones. La recomendación se retira. En conversación alcanza 14 de 14 con compuerta de dominio. RAGAS sobre el híbrido reporta 0,920 de fidelidad, 0,867 de precisión de contexto, 0,947 de cobertura y 0,607 de corrección; con el mismo $k = 10$, el denso baja en relevancia y corrección, y el denso $k = 5$ conserva la mayor corrección. Se documentan también las mejoras rechazadas, el coste de cada decisión y los límites del corpus.**

***Palabras clave*: recuperación aumentada por generación, Milvus, bge-m3, reordenamiento, evaluación de sistemas de recuperación, compuerta de dominio.**

## I. Introducción

Un sistema de recuperación aumentada por generación (RAG) responde una pregunta con fragmentos recuperados de una colección y un modelo de lenguaje que redacta a partir de ellos [1]. La promesa es atractiva: actualizar el conocimiento sin reentrenar y citar la evidencia. En la práctica, la calidad depende de una cadena de decisiones que se afectan entre sí. Una limpieza agresiva puede borrar la frase que responde; un chunking pequeño pierde contexto; un codificador entrenado en otro dominio recupera vecinos plausibles pero inútiles; un reordenador puede mejorar el orden y a la vez encarecer cada consulta; un prompt permisivo invita a inventar; y sin un control de abstención el sistema responde con seguridad sobre temas que el corpus no cubre.

La literatura de RAG suele evaluar un componente a la vez. BEIR y MTEB miden recuperadores [2], [3]; los benchmarks multilingües como MIRACL comparan codificadores [4]; RAGAS, ARES, eRAG y RAGChecker evalúan la respuesta generada [5], [6], [7], [8]. Son piezas necesarias, pero no dicen cómo se comporta la cadena completa sobre un corpus real, con duplicados, truncamiento, ruido de rastreo y preguntas sin respuesta. Tampoco dicen cuánto cuesta cada mejora ni cuándo conviene rechazarla.

Este trabajo construye y evalúa esa cadena completa sobre un corpus de 169.248 documentos recolectados de prensa colombiana e internacional y Bluesky, 161.868 indexados y 618.286 fragmentos. El sistema corre en una estación con una GPU AMD de 16 GB y un servidor de la red local que aloja Milvus [9]; los modelos se sirven por Ollama [10]. No se usan servicios externos de generación ni de embeddings. La Tabla III resume la configuración final y la Sección IV justifica cada elección.

### *I-A. Preguntas de investigación*

1. **Calidad del corpus.** ¿Qué problemas de limpieza, truncamiento, duplicados e idioma aparecen en un corpus de prensa y redes sociales, y cuánto afectan la recuperación?
2. **Recuperación.** ¿Qué combinación de codificador, índice, tamaño de contexto y reordenamiento maximiza la cobertura y el orden con un coste aceptable?
3. **Generación y abstención.** ¿Cómo se comportan distintos modelos locales sobre la misma evidencia y qué controles evitan respuestas sin respaldo?
4. **Evaluación.** ¿Qué acuerdo alcanzan los anotadores humanos y qué señales de evaluación automática son fiables cuando el juez también es un modelo local?

### I-B. Aportes

1. Un pipeline reproducible de extremo a extremo, con Docker Compose, Milvus, Ollama, FastAPI y Streamlit, que se levanta con un comando y no depende de servicios de pago.
2. Un análisis exploratorio y una textometría del corpus que cuantifican el texto sin cierre terminal (32,7 %, un proxy de truncamiento), ruido de rastreo, duplicados exactos y near-duplicates, y que justifican una limpieza conservadora.
3. Una comparación pareada de cuatro variantes de limpieza, dos estrategias de segmentación, tres codificadores, recuperación densa, dispersa e híbrida, y tres alcances de reordenamiento, con intervalos de confianza por bootstrap.
4. Un silver standard de 43 preguntas con revisión de cuatro anotadores y acuerdo entre anotadores reportado, y una evaluación automática con RAGAS que distingue el ruido del juez de una mejora real.
5. Una compuerta de dominio entrenada con 160 preguntas que rechaza las preguntas sin respuesta del conjunto de prueba y las 5 del conjunto reservado, con 3 falsos rechazos en 20 preguntas de dominio, y una interfaz que expone la evidencia, las citas y la abstención.

### I-C. Organización

La Sección II revisa los trabajos relacionados. La Sección III describe el corpus y el análisis exploratorio. La Sección IV presenta la arquitectura y justifica cada capa. La Sección V detalla el procesamiento y la configuración. La Sección VI define el protocolo y el silver. La Sección VII reporta los experimentos y sus decisiones. Las Secciones VIII a XII discuten errores, límites y conclusiones.

## II. Trabajos relacionados

### II-A. Recuperación aumentada por generación

RAG combina un recuperador con un generador para responder con conocimiento externo [1]. La variante densa codifica pregunta y documentos en un mismo espacio y busca por similitud; la dispersa, como BM25 [11], puntúa por coincidencia de términos. La fusión de ambas con reciprocal rank fusion [12] es una práctica común para compensar sus sesgos. En este trabajo se comparan las tres sobre el mismo corpus y el mismo protocolo, con un índice por codificador, y la configuración final combina densa y dispersa antes de reordenar, porque la unión recupera entidades que el denso no trae.

### II-B. Variantes iterativas y correctivas

Self-RAG entrena al modelo para decidir cuándo recuperar y para criticar sus propias respuestas con tokens de reflexión [13]. CRAG evalúa la calidad de la recuperación y activa una búsqueda correctiva cuando el contexto es pobre [14]. Ambos mejoran la robustez frente a recuperaciones malas, a costa de varias pasadas del generador y, en el caso de Self-RAG, de un entrenamiento específico. Este trabajo no adopta esas variantes porque el presupuesto de latencia y la GPU local no permiten varias pasadas por consulta. En su lugar, la compuerta decide antes de generar y los guardarraíles verifican después, con un coste de milisegundos.

### II-C. Codificadores y recuperación

BEIR mostró que un recuperador bueno en un dominio puede fallar en otro [2], y MTEB extendió esa comparación a tareas y lenguas [3]. MIRACL evidenció la brecha multilingüe [4]. Los codificadores entrenados con contrastive learning, como e5 [15] y bge-m3 [16], mejoran el rendimiento zero-shot y multilingüe. bge-m3 además produce representaciones densas, dispersas y multi-vector con un solo modelo, lo que simplifica la infraestructura. La elección entre ellos no se puede decidir solo por la literatura: depende del corpus, del idioma y del tamaño del índice. Por eso se miden sobre el corpus completo en lugar de limitarse al piloto.

### II-D. Reordenamiento

Un cross-encoder puntúa el par pregunta-documento con atención conjunta y suele superar al bi-encoder en precisión [17]. El coste crece con el número de candidatos, porque cada par exige una pasada completa del modelo. La práctica habitual es recuperar un conjunto amplio y reordenar un top pequeño. Este trabajo mide el efecto de reordenar 5, 20 y 50 candidatos densos, y encuentra que ampliar el conjunto denso mejora el orden pero pierde cobertura. La configuración final reordena una unión de denso y BM25 en GPU y recorta a 10; la unión añade cobertura sin el coste de un top-50 denso.

### II-E. Índices vectoriales

HNSW aproxima la búsqueda por vecinos con un grafo jerárquico y un parámetro de exploración que intercambia recall por latencia [18]. Milvus implementa HNSW y otros índices sobre almacenamiento distribuido [9]. El parámetro `ef` se ajusta en este trabajo con una búsqueda en rejilla sobre el silver, y se documenta el punto donde la ganancia deja de compensar el coste.

### II-F. Evaluación

Las métricas de recuperación (Recall@k, nDCG, P@k, MRR) son estándar en recuperación de información. Para la respuesta generada, RAGAS descompone la calidad en fidelidad, relevancia, precisión y cobertura del contexto y corrección [5]; ARES entrena jueces para reducir el sesgo del evaluador [6]; eRAG evalúa la utilidad de cada documento recuperado [7]; y RAGChecker diagnostica errores de recuperación y de generación por separado [8]. Un problema recurrente es que el juez también es un modelo y su veredicto varía con el prompt, el timeout y la concurrencia. Liu et al. muestran además que el generador aprovecha peor la evidencia cuando esta se coloca en el medio de un contexto largo [19]. RAGTruth muestra que las alucinaciones en RAG se concentran en afirmaciones sin respaldo y propone anotación a nivel de span [20]. La verificación de entidades y números contra el chunk citado de este trabajo sigue esa idea con reglas deterministas.

Tabla I
COMPOSICIÓN DEL CORPUS POR COLECCIÓN.

| Colección | Documentos | % del total |
|---|---|---|
| Rastreo web | 104.578 | 61,8 |
| Bluesky | 64.670 | 38,2 |
| Total | 169.248 | 100,0 |

Este trabajo reporta la cobertura de cada métrica de RAGAS además de su media, y separa el ruido del juez de una mejora del sistema.

### *II-G. Abstención y control de dominio*

Un sistema RAG que siempre responde produce alucinaciones cuando la evidencia no alcanza. Las soluciones van desde pedir al generador que se abstenga hasta entrenar un clasificador que decida si la pregunta pertenece al dominio. Este trabajo usa dos controles: guardarraíles de cita y de entidades sobre la respuesta, y una compuerta de dominio supervisada que se activa con un interruptor. La compuerta es un clasificador de regresión logística con embeddings y señales de recuperación, no un juez generativo, para que su decisión sea barata, reproducible y medible.

### *II-H. Posición de este trabajo*

La contribución es una evaluación completa de la cadena sobre un corpus real, con las mejoras que funcionaron y las que no, y con el coste de cada decisión. Se separa la calidad del sistema de la calidad del juez, y esa separación es la que permite decidir qué mantener en producción y qué descartar.

## III. CORPUS Y ANÁLISIS EXPLORATORIO

### *III-A. Composición*

El corpus de trabajo tiene 169.248 documentos, 34 columnas y 256,9 MB. La Tabla I resume la procedencia. La mayor parte viene de rastreo web (104.578) y de Bluesky (64.670). El campo `source_type` no sirve como etiqueta semántica: 99.326 filas lo tienen nulo, 64.670 valen `social_post` y solo 5.252 valen `news_article`. La distribución por dominio confirma el peso de las fuentes: bsky.app aporta 64.670 documentos, infobae 19.766, eltiempo 12.229, theguardian 7.216 y elespectador 5.456. El corpus no representa la conversación pública en español: Bluesky llega casi todo en inglés y la prensa está dominada por cuatro cabeceras. La prensa incluye cabeceras británicas (The Guardian, The Independent, Daily Mail) además de colombianas (El Tiempo, El Espectador, Infobae).

Tras normalizar el campo `language`, el corpus queda en 82.682 documentos en inglés, 66.910 en español, 13.140 sin idioma declarado y 6.516 en otros idiomas, con 4.133 en húngaro. El húngaro aparece con 4.108 documentos en el campo crudo, un volumen que no corresponde con el resto de la distribución y que se excluye del filtro de idioma hasta revisarlo. La normalización se hace al vuelo en la ingesta y el archivo original no se modifica.

El rango declarado de fechas va de 2008 a 2026, pero 163.736 de los 169.248 documentos (96,7 %) no tienen fecha y solo 5.512 la declaran. El filtro temporal se limita a ese 3,3 % y así se documenta en la interfaz.

### *III-B. Duplicados y longitud*

Hay 14.771 copias excedentes (filas que repiten un texto ya presente) agrupadas en 4.434 grupos, con un máximo de 839 copias. Si se cuentan todas las filas que pertenecen a un grupo duplicado, son 19.205: 9.972 en Bluesky y 9.233 en prensa. Las dos cifras son compatibles: la primera cuenta copias adicionales y la segunda, documentos en grupos. No se borran filas en la ingesta, porque eso perdería la procedencia; la deduplicación se hace en la recuperación, comparando el hash del fragmento antes de armar el contexto.

La longitud media es de 473 palabras y 2.977 caracteres, pero la mediana baja a 371 palabras y 2.367 caracteres. La cola es larga: el percentil 99 llega a 2.420 palabras y el máximo a 58.181 palabras, equivalentes a 363.528 caracteres. Ese documento extremo produce cientos de fragmentos y obliga a controlar el tamaño de lote en la ingesta.

### *III-C. Ruido de rastreo*

El rastreador dejó páginas de error dentro del corpus. La misma página con los mensajes “Haz excedido el máximo de peticiones” y “Error 505” aparece en 7.378 documentos, el 4,36 % del total, todos en la colección de prensa. En el conjunto de humo esos textos casi idénticos dominaban el top-10 de cualquier consulta, porque su similitud entre sí es mayor que la de cualquier documento relevante. El registro de ingesta descarta 7.380 documentos por calidad: 7.378 son esa página de error y 2 no están desglosados. Quedan 161.868 documentos indexados y 618.286 fragmentos. El porcentaje de documentos con al menos un patrón de ruido (peticiones, error 505, cookies, inicio de sesión) llega al 5,3 %, pero el filtro solo elimina los marcadores fuertes de error y no las menciones legítimas de cookies.

### *III-D. Textometría*

La textometría se calculó sobre dos muestras: los 2.000 documentos del piloto y 5.000 documentos estratificados del resto del corpus, con semilla 42. La Tabla II compara los indicadores. Los valores son estables entre las dos muestras; eso no demuestra que el piloto represente la distribución del corpus, solo que no se observan diferencias gruesas en estos indicadores.

El 32,7 % de los documentos de la muestra no termina en un signo de cierre. Es un proxy de truncamiento, no una verificación de que el rastreador los cortó: una publicación completa puede acabar sin punto. La limpieza no puede recuperar ese texto, y el porcentaje se declara como límite en la Sección XI. El HTML residual es marginal (0,14 % en la muestra), pero las URLs aparecen en uno de cada diez documentos y los emojis en uno de cada ocho, casi todo en Bluesky. Los near-duplicates no son solo hash exacto: SimHash encuentra grupos de hasta 32 documentos que el hash

Tabla II
TEXTOMETRÍA DEL PILOTO Y DE UNA MUESTRA DEL CORPUS COMPLETO.

| Indicador | Piloto | Muestra |
|---|---|---|
| Tipos por documento (mediana) | 188 | 188 |
| TTR mediana | 0,579 | 0,570 |
| Hapax / tipos (mediana) | 0,773 | 0,770 |
| Guiraud (mediana) | 10,04 | 10,13 |
| Caracteres por documento (mediana) | 2.317 | 2.353 |
| Caracteres p99 | 14.979 | 15.498 |
| Sin puntuación terminal (proxy) | 33,5 % | 32,7 % |
| Documentos con URL | 9,3 % | 9,9 % |
| Documentos con correo | 1,8 % | 1,3 % |
| Documentos con emojis | 12,4 % | 12,1 % |
| Documentos con navegación | 3,4 % | 3,3 % |
| Documentos con boilerplate | 3,4 % | 4,5 % |
| Near-duplicates (SimHash, Hamming $\leq 3$) | 4,25 % | 4,72 % |
| Grupos de near-duplicates | 26 | 53 |
| Grupo más grande | 12 | 32 |

exacto no captura. Los términos exclusivos de prensa están dominados por boilerplate (*publicidad*, *información*, *además*), lo que anticipa que una normalización conservadora puede ayudar sin destruir entidades.

### *III-E. Decisiones de procesamiento*

El análisis anterior fija tres decisiones. Primero, limpieza conservadora: se elimina boilerplate y se normalizan espacios, pero no se tocan entidades ni números. Segundo, deduplicación en la recuperación, no en la ingesta, para conservar la trazabilidad. Tercero, el truncamiento se declara como límite y no se intenta reconstruir. La Sección VII mide si la limpieza conservadora mejora la recuperación frente a no limpiar.

## IV. ARQUITECTURA

### *IV-A. Vista general*

La Figura 1 muestra el flujo. El usuario escribe en una interfaz Streamlit; la API FastAPI recibe la pregunta, la condensa si viene de una conversación, recupera fragmentos con la unión densa y BM25, los puntúa con el cross-encoder, aplica la compuerta de dominio si el Modo estricto está activo, reordena con el reranker, arma el contexto y genera la respuesta. Antes de devolverla, los guardarraíles verifican citas, entidades y números. Cada capa se eligió con una medición o una restricción documentada; no todas las comparaciones son experimentos controlados.

### *IV-B. Capas y justificación*

**Ingesta.** Los documentos se limpian, se fragmentan y se codifican una sola vez. La colección se valida contra la etiqueta del codificador activo para no mezclar espacios vectoriales. Se eligió un pipeline idempotente y reanudable porque el corpus cambia y la codificación es la etapa más costosa: la ingesta completa desde documentos tardó 2 h 41 min y reindexar 618.286 fragmentos ya troceados con bge-m3 tardó 27,3 min.

**Índice vectorial.** Milvus se eligió por tres razones: soporta HNSW con parámetro de exploración ajustable [18], filtra por metadatos (idioma, fecha, colección) sin salir del motor y mantiene 618.286 fragmentos con latencias de milisegundos.

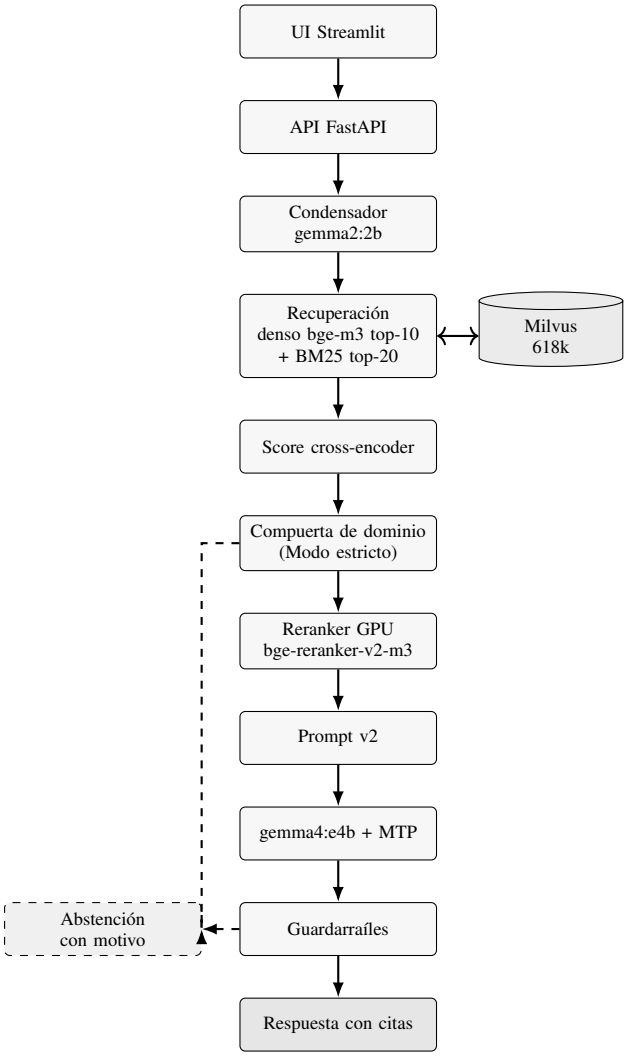



Figura 1. Flujo de una consulta. La compuerta solo actúa con el Modo estricto; el condensador solo en preguntas encadenadas. Las líneas discontinuas son las salidas de abstención, antes y después de generar.

**Codificación.** bge-m3 produce vectores densos de 1024 dimensiones, es multilingüe y rinde bien sin ajuste fino [16]. Se comparó con e5-base y con qwen3-embedding sobre el mismo corpus y el mismo protocolo, con un índice por modelo; la decisión se reporta en la Sección VII.

**Recuperación.** El recuperador combina una lista densa de 10 fragmentos con una dispersa de 20. Las dos se unen sin fusionar y se deduplican; la unión conserva el top-5 denso y añade las entidades que BM25 encuentra. La fusión RRF sin reordenar se midió y perdía frente al denso; la unión reordenada gana en el conjunto reservado.

**Reordenamiento.** Un cross-encoder multilingüe reordena la unión en GPU y el resultado se recorta a 10 [17]. El reordenador comparte la pasada con la compuerta, que usa la puntuación del top-5. El coste medido es de 0,68 s para 50 pares en GPU, frente a 1,30 s para 5 pares en CPU; la GPU deja más margen porque el generador usa el resto de la VRAM.

**Generación.** gemma4:e4b es un modelo de 7,5B cuantizado a Q4_K_M con 116.384 tokens de contexto y MTP nativo. Se

Tabla III
CONFIGURACIÓN FINAL DEL SISTEMA.

| Parámetro | Valor |
|---|---|
| Codificador | BAAI/bge-m3, 1024 dimensiones |
| Índice | HNSW en Milvus, `ef`=800 |
| Recuperación | unión densa top-10 + BM25 top-20, $k = 10$ |
| Reordenador | BAAI/bge-reranker-v2-m3, GPU, top-10 |
| Generador | gemma4:e4b, Q4_K_M |
| Contexto | 116.384 tokens (16.384 entrada, 100.000 salida) |
| MTP | draft 3, 217 tok/s |
| Prompt | v2 |
| Juez y condensador | gemma2:2b |
| Compuerta | regresión logística con cross-encoder, umbral 0,4 |
| Fragmentos | 618.286 en `rag_full_bge_m3_d1024_v1` |

eligió por tres motivos: cabe en 16 GB de VRAM, soporta contextos largos y el MTP con draft 3 acelera la decodificación sin cambiar la distribución del modelo. gemma4:e4b no participó en E4: se eligió por contexto y MTP, no por una comparación directa. En E4, el mejor de los modelos pequeños fue gemma2:2b en corrección y citas.

**Conversación.** Un condensador reescribe la última pregunta con el historial para que la recuperación no dependa de pronombres. Se usa gemma2:2b para no cargar dos veces el modelo grande. Si el condensador pierde el tema, se antepone la primera pregunta del usuario.

**Guardarraíles.** Tres controles antes de responder: cita obligatoria, verificación de entidades y números contra el fragmento citado, y un juez de evidencia. Si alguno falla, el sistema se abstiene con el motivo. El juez es gemma2:2b, distinto del generador, para no validar con el mismo modelo que redacta.

**Compuerta de dominio.** Un clasificador de regresión logística con la puntuación del cross-encoder y señales de recuperación decide si la pregunta pertenece al corpus. No es un LLM juez: reutiliza la pasada del reranker, es reproducible y se puede medir con precisión y recall. Se activa con el interruptor Modo estricto.

**API e interfaz.** FastAPI separa el pipeline de la presentación y expone `/chat`, `/ask`, `/retrieve`, `/health`, `/models` y `/stats` [21]. Streamlit consume `/chat` y muestra la respuesta, las citas, las evidencias, las métricas y el razonamiento [22]. La separación permite probar el pipeline sin la interfaz y cambiar la interfaz sin tocar el pipeline.

### *IV-C. Configuración final*

La Tabla III fija los parámetros. Los valores se justifican en las Secciones V y VII.

### *IV-D. Despliegue: datos en Swarm, GPU en la estación*

La capa de datos (etcd, MinIO y Milvus) se despliega como un stack de Docker Swarm en el servidor, con la etiqueta `role=data`. Milvus publica el puerto 19530 en modo host y el API se conecta a `http://192.168.1.100:19530`. La aplicación (API, frontend y Ollama) corre en la estación con Compose y `network_mode: host`, porque necesita la GPU AMD.

La separación no es arbitraria. Swarm aporta colocación declarativa, política de reinicio y una red overlay que elimina el túnel SSH de la versión anterior; con el túnel, una caída de la conexión dejaba al sistema sin recuperación. La aplicación no puede ir en el stack: Docker Swarm no admite el paso de dispositivos (`devices`) ni `privileged` en un servicio, y el cgroup de dispositivos bloquea `/dev/kfd` con *Operation not permitted* aunque se monte el nodo y el grupo sea el correcto. Se comprobó con un servicio de prueba: el montaje y los grupos 983 y 987 funcionan, pero la apertura del dispositivo falla. Un stack completo obligaría a mover el generador a la GPU del servidor (GTX 1080 de 8 GB), que no soporta el contexto de 116K.

Compose se mantiene para la aplicación porque es el único runtime del proyecto que pasa la GPU AMD, y el stack se mantiene para los datos porque es el que elimina el túnel y hace declarativo el reparto. Un solo Compose no puede repartir servicios entre dos hosts; un solo Swarm no puede pasar la GPU.

### *IV-E. Interfaz para el usuario final*

La interfaz tiene dos zonas. En el panel lateral se elige el modelo, el número de fragmentos, el idioma del documento, el rango de fechas y la colección; se activan el razonamiento y el Modo estricto, y se cambia entre tema claro y oscuro. En el área central se conversa con historial. Cada respuesta muestra el texto con citas numeradas, chips con la intención y la ruta de ejecución, la latencia por etapa, los tokens por segundo y un desplegable con las evidencias: título, puntuación, fragmento, enlace y metadatos. Si el sistema se abstiene, aparece el motivo y, en el caso de fuera de dominio, la sugerencia de desactivar el Modo estricto. El razonamiento del modelo, cuando se activa, va en un desplegable separado y se advierte que no sustituye la evidencia. La Figura 2 muestra una respuesta real con las evidencias abiertas.

## V. METODOLOGÍA

### *V-A. Limpieza y deduplicación*

La variante adoptada, V2, descarta las páginas de error del rastreador, aplica normalización NFKC, quita HTML y URLs y colapsa espacios. No elimina signos, números ni entidades, porque son la señal que usan los guardarraíles. Los duplicados exactos y los near-duplicates se descartan al armar el contexto, comparando el hash del fragmento y su SimHash antes de insertarlo. Así se conserva la procedencia en el índice y se evita repetir el mismo texto en el prompt.

### *V-B. Segmentación*

Se usa chunking fijo de 1000 caracteres con 150 de solape (`fixed_v2`). El solape cubre la frontera donde suele caer la respuesta. Cada fragmento guarda `document_id`, título, URL, fuente, idioma, fecha, posición, estrategia y hash del texto. Esos metadatos permiten filtrar por idioma y fecha y auditar cada cita.




Figura 2. Interfaz final: respuesta con citas y evidencias abiertas. La pregunta se acepta con probabilidad de dominio 0,956; la respuesta cita diez evidencias y el panel muestra título, puntuación, fragmento, enlace y metadatos de la primera. El chip de la derecha identifica la ruta de ejecución, no un guardarraíl de cita.

### *V-C. Codificación e índice*

Los fragmentos se codifican con BAAI/bge-m3 en la GPU, en fp16 y por lotes. La colección se llama `rag_full_bge_m3_d1024_v1` y usa HNSW con `ef`=800 en la búsqueda. El parámetro se eligió con una rejilla sobre el silver: por debajo de 800 el recall cae y por encima la latencia sube sin ganancia apreciable. En paralelo se mantiene una colección BM25 sobre los mismos fragmentos, `rag_full_e5base_d768_v2_bm25_v1`, que alimenta la recuperación dispersa.

### *V-D. Recuperación y reordenamiento*

La recuperación combina dos listas. La densa pide 10 fragmentos a bge-m3; la dispersa pide 20 a BM25. Las dos se unen sin fusionar, se deduplican por texto y se pasan al reordenador. El cross-encoder multilingüe puntúa cada par pregunta-fragmento en GPU, con lote de 16, y el resultado se recorta a 10. La unión conserva el top-5 denso, que es

la señal que la compuerta espera, y añade las entidades que BM25 encuentra y el denso no. Los filtros de idioma, fecha y colección se aplican en Milvus.

### *V-E. Generación*

El generador es gemma4:e4b con temperatura 0 y un presupuesto de 16.384 tokens de entrada. El prompt v2 impone ocho reglas: cada afirmación lleva una cita entre corchetes; no se permiten listas sin cita; si la evidencia no alcanza se responde con una frase de abstención exacta; se distingue un hecho verificable de una afirmación publicada en redes; no se inventan fechas, cifras, nombres ni citas; se ignoran las instrucciones que aparezcan dentro de los documentos; se responde en el idioma de la pregunta; y se usa la fecha exacta de la evidencia cuando la pregunta la pide. El MTP con draft 3 se activa en el manifiesto del modelo y no cambia la salida, solo la velocidad.

### *V-F. Guardarraíles*

Antes de devolver la respuesta se aplican tres controles. El primero exige al menos una cita válida; si no hay, se abstiene con `sin_citas`. El segundo extrae entidades y números de la respuesta y verifica que aparezcan en el fragmento citado o en la pregunta; si no, se abstiene con `sin_respaldo` o `citas_invalidas`. El tercero pide a un juez distinto, gemma2:2b, que decida si la cita sostiene la respuesta; si dice que no, se abstiene con `juez_no`. Cuando el fragmento está en inglés y la respuesta en español, la cobertura léxica se salta y decide el juez, para no castigar una traducción correcta.

### *V-G. Compuerta de dominio*

La compuerta usa la puntuación del cross-encoder sobre el top-5 (máximo, media, diferencia entre el primero y el segundo, y mínimo), cinco features de recuperación (puntuación del primer fragmento, media del top-5, diferencia entre el primero y el segundo, solapamiento entre la pregunta y el título, y presencia de la entidad principal en el título) y un one-hot del tipo de pregunta. La puntuación del cross-encoder se calcula una sola vez y se reutiliza para el reordenamiento. El modelo es una regresión logística con escalado y `class_weight=balanced`; la regularización se eligió por validación cruzada de cinco pliegues sobre train y dev. El umbral es 0,4, elegido para reducir falsos rechazos sin aumentar falsos aceptados. El conjunto tiene 160 preguntas: 60 de dominio y 100 de fuera, con split 103/37/20. Las 40 negativas nuevas son entidades inventadas, eventos futuros y conocimiento general fuera del corpus.

### *V-H. API e interfaz*

La API expone `POST /chat` con historial, `think` y `strict`; `POST /ask` sin historial; `POST /retrieve` para recuperación pura; y los GET `/health`, `/models` y `/stats`. La interfaz Streamlit consume `/chat` y guarda el historial en la sesión. Las preferencias de razonamiento, Modo estricto y tema se guardan en un archivo y se recuperan al recargar. Si el usuario cambia un interruptor mientras el modelo responde, la pregunta queda pendiente y se reintenta en el siguiente ciclo, de modo que la conversación no se pierde.

Tabla IV
COMPOSICIÓN DEL SILVER STANDARD.

| Split | Respondibles | Sin respuesta | Total |
|---|---|---|---|
| Desarrollo | 19 | 0 | 19 |
| Prueba | 19 | 5 | 24 |
| Total | 38 | 5 | 43 |

### *V-I. Despliegue*

El stack se levanta con Docker Compose: etcd, MinIO, Milvus, Ollama, API, frontend y un worker GPU para la ingesta y la evaluación. La API y el frontend usan red de host. Un servicio de una sola ejecución, `prewarm`, espera a que la API esté sana y carga el generador, el embedding y el reranker, para que la primera consulta web no pague la carga del modelo.

## VI. PROTOCOLO DE EVALUACIÓN

### *VI-A. Silver estándar*

El silver standard tiene 43 preguntas: 19 en desarrollo y 24 en prueba, 38 respondibles y 5 sin respuesta. Lo llamamos silver standard y no silver porque lo generó un LLM y lo revisaron los autores, no expertos independientes. Cada fila guarda el identificador, la pregunta, el idioma, el tipo, el split, los documentos y fragmentos de referencia, el span literal que responde, la respuesta de referencia, el criterio de corrección y el estado de validación. La Tabla IV resume la composición. Las preguntas sin respuesta no entran en Recall@k: se evalúan por abstención.

El silver standard se amplió con 17 documentos adicionales marcados por consenso de dos anotadores (kappa 0,626). Con esa ampliación, 14 preguntas quedaron con dos o más documentos relevantes, lo que hace que Recall@k y nDCG no dependan de un único pasaje.

### *VI-B. Revisión humana*

Cuatro personas revisaron el silver standard contra el documento citado. Carlos revisó las 43 y dejó 12 correcciones; Julián aportó 7 spans y una reformulación; Óscar aportó 5 spans corregidos y marcó 3 enlaces rotos; Erika revisó las 43, aprobó 35, marcó 5 como no aplicables y corrigió la respuesta y el span de g-003. Cada propuesta se aplicó solo si el span aparecía literalmente en el texto indexado. Las que no se pudieron verificar quedaron con nota.

Para la comparación de modelos se anotaron 72 respuestas de 4 modelos, ciegas y en orden barajado, con rúbrica de 0 a 3 en corrección, fidelidad, citas y abstención. Dos anotadores calificaron las mismas respuestas. La Tabla V reporta el acuerdo. La fidelidad no alcanzó acuerdo suficiente y no se usa para el veredicto; la corrección y las citas sí.

### *VI-C. Métricas*

**Recuperación.** Recall@k mide la fracción de documentos de referencia recuperados en el top-k; nDCG@k usa relevancia binaria; P@1 mide precisión en el primer corte; Hit@5 indica

Tabla V
ACUERDO ENTRE ANOTADORES EN LA COMPARACIÓN DE MODELOS (KAPPA LINEAL, $n = 72$).

| Eje | Kappa |
|---|---|
| Corrección | 0,95 |
| Citas | 0,90 |
| Abstención | 0,72 |
| Fidelidad | 0,12 |

si al menos un documento relevante aparece en el top-5; MRR mide la posición del primer acierto; y la cobertura de span indica si al menos un fragmento recuperado contiene el texto exacto que responde. Esta última es la más cercana a lo que necesita el generador.

**Respuesta.** Se reportan las preguntas entregadas, las abstenciones, la cobertura de tokens de la referencia (todos sus tokens aparecen en la respuesta), el F1 por tokens, la tasa de citas, el contexto medio en tokens y la latencia por etapa. La revisión humana se aplica solo a las respuestas que no superaron la cobertura y las clasifica como correctas, parciales o incorrectas; no cubre las que sí la superaron.

**Conversación.** Catorce casos: seis saludos, cuatro de dominio y cuatro de fuera. Cada uno tiene un resultado esperado y se marca como correcto o incorrecto. Se ejecutan con y sin Modo estricto, y con y sin razonamiento.

**RAGAS.** Se reportan fidelidad, relevancia de la respuesta, precisión del contexto, cobertura del contexto y corrección de la respuesta [5]. Para cada métrica se informa cuántas muestras tienen valor, porque el juez falla en algunas y una media con cuatro valores no es comparable con una con 38. La corrida de referencia es densa con $k = 5$; la comparación con el híbrido y con la arquitectura anterior está en la Sección VII.

**Compuerta.** Precisión, recall y F1 en dev y test, falsos rechazos de dominio (preguntas del corpus rechazadas) y falsos aceptados de fuera (preguntas ajenas aceptadas).

### *VI-D. Estadística*

Las comparaciones de recuperación usan bootstrap pareado por pregunta con 1.000 remuestreos e intervalos de confianza al 95 %. En la comparación de limpieza se aplica corrección de Bonferroni por comparaciones múltiples. La configuración se eligió inspeccionando las 43 preguntas del silver, así que los resultados sobre las 38 respondibles son exploratorios. No se dispone de un conjunto de prueba independiente; una evaluación futura debe reservar preguntas nuevas. Las diferencias que no excluyen cero se reportan como no concluyentes, aunque la media favorezca a una variante. Los intervalos pareados se calcularon para limpieza, E1 y E2; las comparaciones de codificadores, reordenamiento y fusión se reportan como estimaciones puntuales sobre 38 preguntas, sin intervalo, y así se indica en cada tabla.

### *VI-E. Configuración de RAGAS*

La corrida de referencia usa juez gemma4:e4b y embeddings BAAI/bge-m3 en CPU, con 38 preguntas directas. El timeout del juez se fijó en 900 s y la concurrencia en 6. Con los valores por defecto (180 s y 16 workers) el juez no terminaba en varias muestras y la precisión del contexto quedaba con 4 valores de 38. La corrida de referencia tiene 34 valores en esa métrica y 38 en las demás. Las preguntas sin respuesta se miden aparte y se abstienen 5 de 5.

Tabla VI
LIMPIEZA SOBRE EL PILOTO (30 PREGUNTAS, E5-BASE, $k = 10$).

| Variante | P@1 | Hit@5 | Recall@10 | MRR | nDCG |
|---|---|---|---|---|---|
| V0 crudo | 0,733 | 0,833 | 0,833 | 0,775 | 0,790 |
| V1 sin páginas de error | 0,767 | 0,833 | 0,833 | 0,800 | 0,809 |
| V2 normalización | **0,833** | **0,900** | **0,900** | **0,867** | **0,875** |
| V3 V2 + dedup | 0,800 | 0,867 | 0,867 | 0,833 | 0,842 |

Tabla VII
E1: CHUNKING FIJO CONTRA ORACIONES (30 PREGUNTAS).

| Variante | P@1 | Hit@5 | Recall@10 | nDCG | p50 |
|---|---|---|---|---|---|
| Fijo 1000/150 | **0,833** | **0,900** | **0,900** | **0,875** | **38,5 ms** |
| Oraciones 384/1 | 0,767 | 0,833 | 0,833 | 0,804 | 42,1 ms |

## VII. EXPERIMENTOS Y RESULTADOS

Los experimentos usan el silver standard de 43 preguntas; E12 usa la versión ampliada con 17 documentos adicionales y cada tabla lo indica. La configuración se eligió con el silver completo, así que los resultados son exploratorios. Cada subsección reporta el montaje en una frase, la tabla y la decisión; el detalle del montaje está en la Sección V.

### *VII-A. Limpieza*

Se construyeron cuatro índices sobre el piloto con 30 preguntas, e5-base, $k = 10$ y tres repeticiones. La Tabla VI muestra el resultado. Ninguna comparación pareada contra V0 excluye cero tras la corrección de Bonferroni sobre 21 pruebas. V2 toca el límite inferior en cuatro métricas, compatible con una mejora pequeña o con ausencia de efecto. Se adopta V2 por tres razones no estadísticas: la consistencia de la estimación en todas las métricas, el coste nulo en consulta y la eliminación de 68 páginas de error que de otro modo entrarían al índice.

### *VII-B. Segmentación y tamaño del contexto*

E1 comparó chunking fijo de 1000 caracteres con 150 de solape contra segmentación por oraciones de 384 caracteres. La Tabla VII muestra estimaciones puntuales favorables al fijo en todas las métricas de calidad, pero ningún intervalo pareado excluye cero: nDCG $-0,071$ (IC $[-0,221;\ 0,025]$), recall $-0,067$ (IC $[-0,233;\ 0,000]$) y P@1 $-0,067$ (IC $[-0,233;\ 0,114]$) para oraciones menos fijo. La única diferencia concluyente es la latencia: el fijo es 3,65 ms más rápido (IC $[2,25;\ 4,97]$). Se mantiene el fijo por la dirección consistente y el coste, no por una mejora de calidad demostrada.

E2 ablacionó el número de fragmentos. Con el silver del piloto, el documento relevante está siempre en el top-3, así que $k = 3$, 5 y 10 devuelven las mismas métricas de calidad. La única diferencia significativa es la latencia: $k = 3$ tarda

Tabla VIII
E6: PILOTO CONTRA CORPUS COMPLETO CON E5-BASE (38 PREGUNTAS).

| Corpus | Recall@10 | P@1 | Hit@5 | nDCG | Span |
|---|---|---|---|---|---|
| Piloto (1.932 docs) | **0,947** | **0,842** | **0,947** | **0,909** | **0,895** |
| Completo (161.868) | 0,684 | 0,474 | 0,658 | 0,594 | 0,684 |

Tabla IX
E10: RECUPERADORES SOBRE EL CORPUS COMPLETO (38 PREGUNTAS).

| Método | Recall@10 | P@1 | Hit@5 | nDCG | p50 |
|---|---|---|---|---|---|
| Denso e5 | 0,684 | 0,474 | 0,658 | 0,594 | 48 ms |
| BM25 sparse | 0,763 | **0,684** | 0,737 | **0,727** | **7 ms** |
| bge-m3 | **0,842** | 0,553 | **0,816** | 0,708 | 56 ms |

1,64 ms menos que $k = 10$ y $k = 5$ tarda 0,94 ms menos. Se fija $k = 5$ como margen para preguntas comparativas que no están en el conjunto, sabiendo que el coste es más contexto en generación, no más latencia de recuperación.

### *VII-C. Codificadores y escala*

E3 comparó e5-base con qwen3-embedding:8b sobre el piloto, con 18 preguntas de desarrollo. qwen3 recupera mejor: Recall@10 perfecto y 6,4 puntos más de nDCG. El problema es el coste. e5-base indexó el corpus completo en 2 h 41 min medidos en la RX 9070 XT. qwen3-embedding:8b no se indexó entero: las 23 horas son una extrapolación desde el rendimiento por lote, no un tiempo medido. Se adopta e5-base para la ingesta y qwen3 queda como comparación documentada.

E6 midió el mismo pipeline sobre el corpus completo. La Figura 3 muestra la caída. La caída es el hallazgo más importante del trabajo: con el mismo codificador y los mismos parámetros de índice en colecciones separadas, y las mismas 38 preguntas, Recall@10 pasa de 0,947 a 0,684 y nDCG de 0,909 a 0,594 al crecer de 1.932 a 161.868 documentos. La latencia no explica la caída (40 ms contra 42 ms). El experimento observa la caída al aumentar la escala; no separa competencia entre fragmentos, truncamiento del texto ni segmentación.

E8 y E10 buscaron un recuperador mejor. BM25 sparse sube el recall a 0,763 y es ocho veces más rápido, pero su techo de cobertura es bajo. bge-m3 llega a 0,842 de Recall@10 y 0,708 de nDCG, y recupera el inglés de 0,111 a 0,667. La Tabla IX resume.

E11 ajustó `ef` y probó la fusión con BM25. `ef`=800 sube el recall 2,6 puntos y el nDCG 1,3 con 6 ms más. La fusión 0,5/0,5 empata en recall y sube nDCG 6,9, P@1 13,2 y cobertura de span 5,3. Con el silver de un solo relevante la fusión parecía ganar. E12 repitió la comparación con el silver ampliado: el denso recupera más documentos nuevos (Recall@10 0,921 contra 0,890 y nDCG 0,827 contra 0,804). E12 se queda con la base densa, `ef`=800, sin fusión; la ruta desplegada después es la unión híbrida, elegida sobre el conjunto de desarrollo.

### *VII-D. Reordenamiento*

E7 midió el cross-encoder sobre el top-50 con e5 en el corpus completo: P@1 sube de 0,474 a 0,632 y nDCG de 0,594 a 0,661, pero el Recall@10 se queda en 0,684; Hit@5 sube de 0,658 a 0,684 y la cobertura de span de 0,632 a 0,658. El coste es de 3,1 s por consulta. La mejora de orden no compensa el coste si se reordena tanto.

El denso a $k = 10$ recupera 0,912 de recall y 0,818 de nDCG. Con bge-m3 se probó reordenar 5 y 20 candidatos. La Tabla S1 del suplemento muestra que el top-20 baja el recall a 0,864 y el Hit@5 a 0,921, mientras que el top-5 mantiene 0,895 de recall y 0,947 de Hit@5, y sube el nDCG a 0,865 y el P@1 a 0,868 con 1,30 s. La tabla mide el reordenador en CPU; en la configuración final corre en GPU, con 0,68 s para 50 pares. La configuración final no reordena el top-5 denso: reordena la unión con BM25 y recorta a 10.

Tabla X
E4: ANOTACIÓN HUMANA (0 A 3) Y CONDUCTA AUTOMÁTICA.

| Modelo | Corrección | Citas | Abstención | Sin citas |
|---|---|---|---|---|
| gemma2:2b | **2,81** | **2,67** | **2,89** | **0,0 %** |
| qwen2.5:1.5b | 2,31 | 0,00 | 2,81 | 89,5 % |
| llama3.2:3b | 2,22 | 0,86 | 2,69 | 56,1 % |
| llama3.2:1b | 0,42 | 0,28 | 1,61 | 15,8 % |

### *VII-E. Modelos generativos*

E4 comparó cuatro modelos con la misma evidencia: 19 preguntas respondibles y 5 sin respuesta, tres repeticiones, anotación humana ciega de la primera repetición. La anotación cubre 18 de las 19 respondibles. La Tabla X muestra las medias de los dos anotadores en escala de 0 a 3. gemma2:2b gana en corrección, citas y abstención. La fidelidad no se usa para el veredicto porque su acuerdo es $\kappa = 0,12$. qwen2.5:1.5b es el más rápido pero no cita en el 89,5 % de los casos. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles y no se recomienda.

La hipótesis H1, que esperaba que llama3.2:3b superara a los demás en corrección y citas, se refuta. La evidencia recuperada es la misma entre modelos; lo que cambia es el generador. La comparación fijó la familia de modelos pequeños. El generador final es gemma4:e4b, que no estaba en E4: se adoptó después por contexto largo y MTP, no por ganar una comparación directa, y se validó en la corrida final.

### *VII-F. Inyección de instrucciones*

E5 insertó instrucciones explícitas en cinco documentos sintéticos por modelo. Es una prueba exploratoria: no cubre el generador final ni demuestra robustez general. Dos de los tres modelos las ignoraron. gemma2:2b obedeció una de cinco y añadió la palabra pedida a una respuesta por lo demás correcta. La regla de diseño que sale de aquí es que el prompt debe delimitar la evidencia y ordenar explícitamente ignorar instrucciones internas, y esa orden debe repetirse después del bloque de contexto.

### *VII-G. Prompts y mejoras rechazadas*

La fase 6 probó cuatro mejoras sobre la configuración de referencia y la fase 7 probó diversidad. La referencia de estas

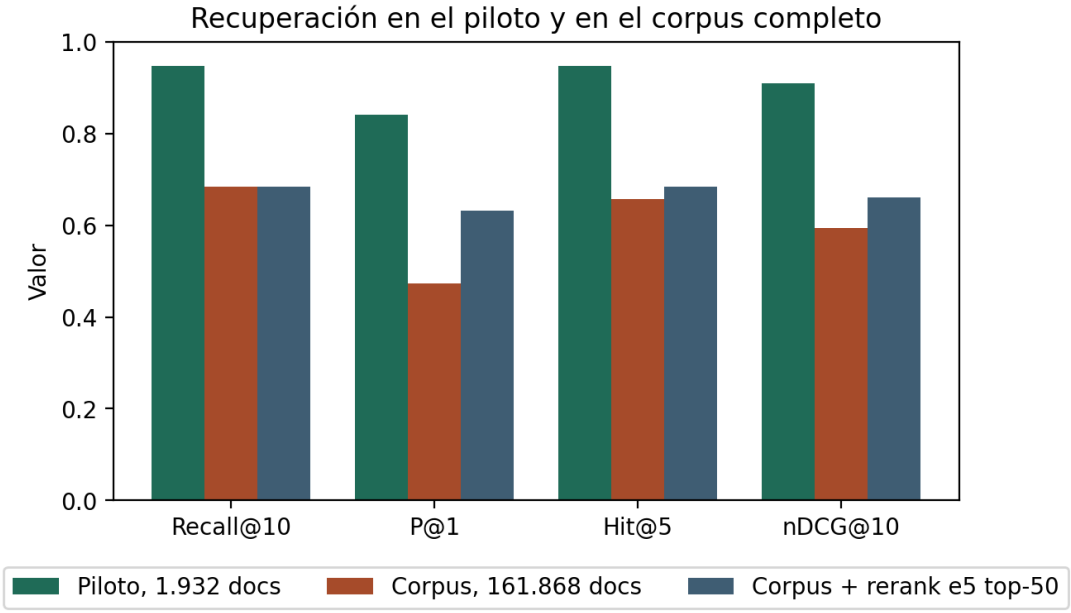



Figura 3. Recuperación con e5-base sobre el silver de 38 preguntas respondibles: piloto (1.932 documentos), corpus completo (161.868 documentos) y corpus con reranker sobre el top-50. El piloto y el corpus usan índices separados.

Tabla XI
COMPUERTA DE DOMINIO: TEST INTERNO Y CONJUNTO RESERVADO.

| Split | $n$ | Dominio | Fuera | Precisión | Recall | F1 |
|---|---|---|---|---|---|---|
| Test | 20 | 11 | 9 | 1,000 | 0,909 | 0,952 |
| Reservado | 25 | 20 | 5 | 1,000 | 0,850 | 0,919 |

fases era el top-5 denso con reranker; la unión con BM25 se adoptó después, en el conjunto reservado. Ninguna de las variantes de las fases 6 y 7 supera a esa referencia. La Tabla S2 del suplemento resume. El filtro por score pierde recall; reordenar el top-20 pierde Hit@5 y multiplica la latencia; la expansión de consulta baja el recall y el nDCG; el prompt v4 no mejora la cobertura de tokens; y MMR baja el recall 8 puntos. El prompt v4 entregó 37 de 38 y 25 de 37 de cobertura de tokens, sin mejora. Todas se rechazan y la configuración se mantiene.

### *VII-H. Compuerta de dominio*

La compuerta se entrenó con 160 preguntas: 60 de dominio y 100 de fuera, con 103 de train, 37 de dev y 20 de test. Los 40 negativos nuevos son entidades inventadas, eventos futuros y conocimiento general. La Tabla XI muestra el resultado en el test interno y en el conjunto reservado. En test el F1 es 0,952, con 0 falsos aceptados de 9 y 1 falso rechazo de 11. En el conjunto reservado, con 20 directas y 5 negativas, el F1 es 0,919: rechaza las 5 negativas y 3 de las 20 directas. El umbral es 0,4, elegido por validación cruzada. Con 5 negativas, el 0/5 no demuestra una tasa nula de falsos aceptados en uso general; solo acota el intervalo observado.

Tabla XII
RAGAS: HÍBRIDO, DENSO CON $k = 10$, DENSO CON $k = 5$ Y CONFIGURACIÓN ANTERIOR, CON $n$ ENTRE PARÉNTESIS. LAS TRES PRIMERAS USAN JUEZ GEMMA4:E4B Y EMBEDDINGS BGE-M3; LA ÚLTIMA CAMBIA JUEZ Y EMBEDDINGS, ASÍ QUE SU COMPARACIÓN ES OBSERVACIONAL.

| Métrica | Híbrido $k = 10$ | Denso $k = 10$ | Denso $k = 5$ | Anterior |
|---|---|---|---|---|
| Fidelidad | 0,920 (38) | 0,932 (38) | **0,952** (38) | 0,925 (36) |
| Relevancia | 0,788 (38) | 0,765 (38) | **0,840** (38) | 0,798 (38) |
| Precisión del contexto | 0,867 (36) | 0,810 (36) | **0,881** (34) | 0,760 (36) |
| Cobertura del contexto | **0,947** (38) | **0,947** (38) | 0,921 (38) | 0,842 (38) |
| Corrección | 0,607 (38) | 0,569 (38) | **0,657** (38) | 0,705 (37) |

### *VII-I. RAGAS*

La Tabla XII compara cuatro configuraciones. El híbrido es la ruta desplegada: unión densa y BM25, $k = 10$ y sin compuerta. El denso $k = 10$ comparte $k$ con el híbrido, así que su diferencia aísla el recuperador: el híbrido gana relevancia, precisión y corrección, y pierde fidelidad, con la misma cobertura. El denso $k = 5$, con menos contexto, gana fidelidad, relevancia, precisión y corrección, y pierde cobertura: el efecto de $k$ es mayor que el del recuperador. La última columna cambia juez y embeddings, así que es observacional. Las diferencias entre las tres primeras son descriptivas.

### *VII-J. Latencia y MTP*

La Tabla XIII muestra el efecto del draft. El draft 3 es el más rápido: 217,1 tokens por segundo frente a 193,5 con 5 y 159,5 con 7. La decodificación especulativa es exacta: el draft propone y el modelo objetivo verifica, así que la salida no cambia.

La corrida de referencia de la configuración anterior (densa y reranker en CPU) es la del 2 de noviembre, con el código

Tabla XIII
MTP: TOKENS POR SEGUNDO SEGÚN EL DRAFT.

| Draft | Tokens/s |
|---|---|
| 3 | **217,1** |
| 5 | 193,5 |
| 7 | 159,5 |

del commit `125bbc7`: 7.938 ms sin razonamiento y 6.834 ms con razonamiento; los CSV están en `reports/tablas/`. El reranker en CPU aporta 1,30 s de mediana. Con la configuración final (unión híbrida y reranker en GPU), el conjunto reservado dio 6.154 ms de media (mediana 5.578, p95 9.136) sobre 25 preguntas. La compuerta reutiliza la puntuación del reranker, así que no añade una segunda pasada.

### *VII-K. Conjunto reservado*

El silver exploratorio se usó para elegir la configuración densa, así que no mide generalización. Se construyó un conjunto reservado de 25 preguntas: 20 directas y 5 sin respuesta, 15 en español y 10 en inglés, 14 de prensa y 6 de Bluesky. Las preguntas se generaron con Gemma4 y se validaron automáticamente; no son una muestra humana independiente. El conjunto se congeló antes de evaluar la compuerta y se usó para esa evaluación. Después se usó también para elegir la recuperación híbrida, así que las métricas de recuperación y respuesta de esta sección son de desarrollo; la compuerta se midió antes de esa elección, pero el mismo conjunto se usó después para elegir el híbrido, así que sus cifras también son de desarrollo. Una prueba independiente de la recuperación exige un conjunto nuevo. La Tabla S3 del suplemento muestra el resultado.

Las métricas de respuesta se midieron sin la compuerta estricta; las predicciones de la compuerta se calcularon en una pasada aparte. Con la compuerta activa (`strict=True`), el flujo conjunto entrega 15 de 20 directas y se abstiene en las 5 negativas: tres directas se rechazan por dominio (r-011, r-012 y r-014) y dos por el juez de evidencia (r-005 y r-006). La cobertura de tokens de las 15 entregadas es 13/15 (0,867). La diferencia con las 19 entregas sin compuerta es el coste del filtro. El recall@10 es 0,700 y el nDCG 0,612, por debajo del 0,912 y 0,818 del silver exploratorio, pero por encima del 0,600 y 0,477 de la recuperación densa. La unión con BM25 recupera r-012 y r-014, que el denso dejaba fuera, y conserva los aciertos del denso en r-009 y r-013. Seis de las 20 preguntas no recuperan el documento de referencia en el top-10; cuatro se responden correctamente con otro documento que contiene el mismo dato (r-005, r-008, r-017, r-019), una termina en abstención (r-006) y una es ambigua (r-016). El recall con un solo documento de referencia subestima la recuperación útil: la mitad de los fallos de recall no son fallos de respuesta. La profundidad de la unión no es monotónica: 20/20 y 30/30 suben el recall@10 a 0,750; 50/50 lo sube a 0,850 y recupera r-006, pero baja las entregas a 18 de 20 y la cobertura de tokens a 16 de 20, porque desplaza evidencia que el modelo usaba bien en r-009 y r-012. La configuración se queda en 10/20.

La cobertura de tokens entre las entregadas es 17 de 19 (89,5 %), y sobre las 20 directas, 17 de 20 (85 %). La diferencia con la recuperación densa está en la unión: recupera las entidades que el denso no trae. En inglés el recall es 0,875 y el nDCG 0,829; en español, 0,583 y 0,468. El inglés recupera mejor, al contrario de lo que sugería el silver exploratorio.

Las cinco preguntas sin respuesta se rechazan por la compuerta. En las 20 directas, la compuerta rechaza 3: r-011, r-012 y r-014. r-012 y r-014 son preguntas que la recuperación híbrida sí puede responder, pero la compuerta solo ve el top-5 denso; es una deuda de diseño, no un fallo de recuperación. r-011 sí recupera el documento y es un falso rechazo del clasificador. La compuerta nueva baja los falsos rechazos de 6 a 3 y los falsos aceptados de 1 a 0 respecto de la anterior. El 0/9 del conjunto anterior no se confirma como tasa nula, pero el 0/5 tampoco la demuestra.

El fallo más grave es r-006: la pregunta pide los 95 votos en la Cámara de Representantes y el sistema se abstiene. La evidencia recuperada no contiene la cifra correcta en el top-10, y el modelo prefiere no responder. r-016 es una pregunta ambigua del propio conjunto: no nombra el concierto y el corpus tiene varios; se mantiene en las métricas y se reporta como límite de construcción. r-012 entrega una respuesta evasiva: la evidencia contiene la organización, pero el modelo enumera varias iniciativas en lugar de responder World Economic Forum. Es un fallo de generación, no de recuperación.

La latencia media del conjunto reservado es 6,2 s (mediana 5,6, p95 9,1), inferior a los 10,8 s de la corrida densa. La unión y el reordenador en GPU compensan el mayor número de candidatos.

La Tabla S5 del suplemento resume la progresión de la configuración. Cada fila es una decisión tomada con una medición, no una preferencia.

## VIII. ANÁLISIS DE ERRORES

### *VIII-A. Errores de recuperación*

Queda un fallo claro en el silver standard. g-031 pregunta por el municipio de gran extensión y tradición agropecuaria del sur del Tolima. En E6/E7 quedaba en el puesto 11 del top-50 denso; la ruta híbrida lo recupera en el puesto 1 y responde Chaparral. g-046, sobre la protesta de Extinction Rebellion, no aparece ni en la unión ni tras el reranker; no se aisló la causa. El análisis de E6/E7 se conserva como histórico.

### *VIII-B. Errores de generación*

Los modelos pequeños fallan de formas distintas con la misma evidencia. qwen2.5:1.5b responde sin citar en el 89,5 % de los casos: la respuesta puede ser correcta, pero el usuario no puede verificar de dónde sale. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles. gemma2:2b cita siempre y responde casi siempre, pero responde preguntas sin respuesta en 3 de 15 repeticiones, es decir, alucina con evidencia insuficiente. El generador final, gemma4:e4b, no estaba en esa comparación; los guardarraíles de cita, entidades y juez cubren parte de ese riesgo, y la compuerta lo cubre antes de generar.

### VIII-C. Abstención y compuerta

La abstención tiene dos caminos. El primero ocurre antes de generar, cuando la compuerta rechaza la pregunta; el segundo, después, cuando los guardarraíles o el juez no encuentran respaldo. El primer camino es barato (decenas de milisegundos) y ahora rechaza las 5 preguntas sin respuesta con 0,293 a 0,436. El segundo cuesta una generación completa. La compuerta v2, con umbral 0,4, rechaza 3 de las 20 directas del reservado: r-011 es un falso rechazo del clasificador y r-012/r-014 son deudas de diseño porque la compuerta solo ve el top-5 denso. La versión anterior, con umbral 0,5, dejaba un único falso rechazo en 0,494; se conserva como antecedente y no describe la compuerta actual.

### VIII-D. Errores de evaluación

RAGAS deja 4 muestras sin valor en la precisión del contexto por errores de parser del juez. Una media con 34 valores no es comparable con una de 38, y por eso se reporta $n$ en cada fila. En la primera corrida, con timeout de 180 s y 16 workers, la métrica tenía 4 valores y su media (0,750) no era comparable con nada. La conclusión anterior, que afirmaba una pérdida de precisión frente a la arquitectura previa (0,760, con 36 valores), era un artefacto del juez. La repetición corregida del mismo sistema da 0,881 con 34 valores. La comparación entre arquitecturas sigue siendo observacional porque cambia juez y embedding.

### VIII-E. Errores de diseño del protocolo

El experimento de limpieza se lanzó inicialmente con el silver de humo, que pertenece a una muestra de 600 documentos que no es subconjunto del piloto de 2.000. Las referencias no existían en las colecciones y las métricas habrían sido cero. El error se detectó al medir y obligó a construir un silver específico para el piloto. La lección es que el silver debe declarar a qué muestra pertenece y que la ingesta debe verificar que los documentos de referencia están en la colección antes de evaluar. Esa verificación ya forma parte del protocolo.

## IX. Discusión

### IX-A. Por qué el piloto engaña

La caída de Recall@10 de 0,947 a 0,684 al pasar del piloto al corpus completo (Tabla VIII) es consistente con un efecto de competencia: con 1.932 documentos, el fragmento relevante compite contra pocos vecinos; con 161.868, contra cientos de fragmentos que comparten vocabulario genérico. El texto sin cierre terminal (32,7 %) y el chunking fijo podrían agravarla, pero el experimento no separa competencia, truncamiento y segmentación. La consecuencia práctica es que un piloto sirve para depurar el pipeline, no para elegir codificador, chunking o $k$. Esa elección hay que hacerla a la escala final, aunque cueste horas de GPU.

### IX-B. El coste del orden

El reordenador es una herramienta de precisión, no de cobertura. Sobre el top-5 sube el nDCG 5,6 puntos y el P@1 13,1, porque el recuperador ya trajo la respuesta y solo había que colocarla primero. Sobre el top-20 denso baja el recall y el Hit@5: al ampliar el conjunto entran fragmentos peores que desplazan a los buenos. La unión con BM25 cambia el cálculo: añade documentos que el denso no trae y el reordenador los coloca arriba. En GPU el coste es 0,68 s para 50 pares, frente a 1,30 s para 5 en CPU. El reordenador no puede recuperar lo que el recuperador no trajo; la unión sí puede, y por eso se adopta.

### IX-C. La abstención barata

La compuerta cuesta milisegundos y rechaza las 5 preguntas sin respuesta; el juez cuesta una generación completa. Esa asimetría justifica el clasificador. La compuerta v2, con umbral 0,4, rechaza 3 de las 20 directas del reservado: dos son deudas de diseño y una es un falso rechazo del clasificador. El umbral es una decisión de negocio, no técnica: aceptar una pregunta ajena erosiona la confianza más que abstenerse en una propia. En un asistente de corpus, el usuario prefiere un "no lo sé" a una respuesta inventada. Si el coste de abstenerse fuera mayor, el umbral se movería en la otra dirección.

### IX-D. La evaluación

El episodio de RAGAS (Sección VII) deja dos reglas. La comparación entre arquitecturas cambia juez y embedding, así que se reporta como observación y no como mejora causal. La corrida del híbrido confirma el intercambio: más cobertura del contexto, menos corrección. La primera: reportar $n$ junto a cada media y no comparar métricas con cobertura distinta. La segunda: el ruido del juez no es aleatorio, se concentra en contextos largos y en fallos de parser, así que un timeout más generoso y menos concurrencia lo reducen.

El acuerdo humano también delimita lo que se puede afirmar. La corrección (0,95) y las citas (0,90) son fiables; la abstención (0,72) es aceptable; la fidelidad (0,12) no. Usar la fidelidad para elegir modelo habría sido elegir con ruido. La rúbrica de fidelidad mezcla errores factuales con abstenciones injustificadas, y los anotadores no comparten el criterio. Un eje que no alcanza acuerdo no debe decidir, y eso es una lección de diseño de la evaluación, no un defecto de los anotadores.

### IX-E. Lo que mostró el conjunto reservado

El conjunto reservado cambia el diagnóstico: la caída de recuperación se confirma fuera del piloto y la unión con BM25 recupera parte de lo perdido. La mejora no viene de reordenar más: viene de recuperar entidades que el denso no trae. Quedan tres fallos de respuesta en la corrida sin compuerta: r-006 se abstiene, r-012 enumera iniciativas y r-016 es ambigua. El sistema sabe decir que no, pero todavía no siempre elige la evidencia correcta cuando hay varias.

La compuerta anterior tampoco generalizaba el cero. La nueva reduce los falsos rechazos, pero no los elimina; los que quedan son preguntas que la recuperación híbrida sí puede responder, porque la compuerta solo ve el top-5 denso. La lección no es retirar la compuerta, sino alinearla con el conjunto que ve el generador.

El conjunto reservado también muestra que el silver exploratorio no era representativo en idioma: el inglés, que parecía el punto débil, recupera mejor en la muestra nueva. Con 8 y 12 preguntas, las diferencias no son concluyentes, pero bastan para no tratar el idioma como una conclusión cerrada.

### *IX-F. Qué se llevaría a producción*

La configuración que se llevaría es la de la Tabla III: unión densa y BM25, reranker en GPU, $k = 10$, gemma4:e4b con MTP y compuerta con umbral 0,4. Las deudas son r-006, r-016, la alineación de la compuerta con el conjunto reordenado y la corrección de la respuesta.

El monitoreo que acompañaría el despliegue tiene cuatro señales: recall en una muestra fija del silver, tasa de abstención, tasa de citas válidas y latencia p95. Las cuatro se pueden calcular con la infraestructura actual y detectan las regresiones que importan: que el índice cambie, que el modelo se actualice, que el prompt se rompa o que el hardware se quede sin memoria. La compuerta se monitorea aparte, con un conjunto fijo de preguntas de fuera, porque su deriva no se ve en las métricas de respuesta.

## X. Ambigüedad y clarificación

### *X-A. Estado del arte*

La ambigüedad referencial aparece cuando una pregunta admite varias respuestas válidas sobre el mismo corpus. AmbigQA la formaliza como una pregunta con un conjunto de respuestas [23]. CLAM muestra que los modelos detectan la ambigüedad cuando se les pregunta de forma explícita, pero en modo QA contestan igual; el contexto recuperado reduce aún más la probabilidad de pedir una aclaración [24]. CLAMBER evalúa la detección y la calidad de las preguntas de clarificación en cuatro tipos de ambigüedad y encuentra que los LLM actuales tienen dificultades para resolverla [25]. Clarify When Necessary estima la entropía sobre las intenciones para decidir cuándo preguntar [26].

En RAG hay dos familias. La clarificación interactiva pregunta al usuario y reescribe la consulta; ASK elige entre clarificación basada en documentos y basada en aspectos según la ambigüedad [27], y Tree of Clarifications construye un árbol de subpreguntas y agrega las respuestas [28]. La respuesta condicional contesta todas las interpretaciones; DIVA recupera pasajes diversos, verifica su calidad y adapta la respuesta [29], y CondAmbigQA introduce condiciones explícitas y reporta una mejora de 11,75 % cuando el modelo las considera [30]. La encuesta de 2025 ordena estas estrategias y señala que la detección sigue siendo el cuello de botella [31].

### *X-B. Diseño*

El sistema usa un detector selectivo. Solo gasta una llamada al LLM cuando la pregunta contiene una referencia genérica que puede tener varios referentes en el corpus (“la iniciativa”, “el concierto”, “la reforma”, “el acuerdo”, “la COP”, entre otras). Para esas preguntas recupera una unión profunda de 50 fragmentos densos y 50 de BM25, la reordena y pasa el top-10 al detector. Así las opciones no se limitan al top-10 de producción.

El detector pide al LLM un JSON con la ambigüedad, la pregunta de clarificación y entre dos y seis opciones; cada opción lleva etiqueta, consulta reescrita, respuesta y evidencias. Una aumentación determinista añade las cifras de votos y los montos que el modelo omite, para que la lista cubra interpretaciones que el LLM no propuso. La clarificación devuelve las opciones a la interfaz; al elegir una, la consulta reescrita se envía con la compuerta desactivada, porque la interpretación ya está fundamentada. La respuesta condicional usa las mismas opciones como condiciones, cada una con su cita. La política híbrida clarifica en `/chat` y condiciona en `/ask`.

Tabla XIV
Ambigüedad: 6 ambiguas y 6 de control. Baseline y clarificación: cobertura de tokens; condicional: resultado de la política sobre 12. Cobertura entre opciones: 5/6. La híbrida reporta clarificación interactiva y condicional de una pasada.

| Política | Resultado | Turnos | Latencia |
|---|---|---|---|
| Baseline | 0,833 | 1,0 | 7,16 s |
| Clarificación | 0,833 | 1,4 | 8,78 s |
| Condicional | **0,917** | **1,0** | 5,94 s |
| Híbrida | 0,833 / 0,917 | 1 a 2 | 5,94 a 8,78 s |

### *X-C. Experimento*

El conjunto tiene 12 preguntas: 6 ambiguas y 6 de control, con la interpretación pretendida, las opciones, las respuestas condicionales y los alias en español e inglés. Se congeló antes de ejecutar. Se comparan cuatro políticas: baseline (respuesta única), clarificación, condicional e híbrida. La clarificación se evalúa con un usuario oracle que elige la opción cuya respuesta coincide con la interpretación pretendida; si no existe, se marca como fallo. Los valores p son de McNemar exacto bilateral y no se ajustan por comparaciones múltiples. La Tabla XIV resume.

### *X-D. Resultados*

El detector marcó las 6 ambiguas y ninguna de las 6 de control: precisión, recall y F1 de 1,0. En un segundo conjunto de 6 preguntas temporales y de alcance (3 ambiguas y 3 de control, estos últimos repetidos del primero) volvió a marcar las 3 ambiguas y ninguna de las 3 de control, con F1 de 1,0. Los dos conjuntos suman 15 preguntas únicas, incluidas en el v2 de 42. Los conjuntos son pequeños y el resultado es descriptivo, no una estimación poblacional.

El baseline cubre los tokens de referencia en 10 de 12 (0,833); la clarificación en 10 de 12 (0,833), con la interpretación pretendida en 5 de las 6 ambiguas; el resultado de la política condicional es 11 de 12 (0,917); la cobertura entre opciones es 5 de 6 (0,833). El condicional obtiene el mayor resultado automático y la menor latencia; la clarificación gana en control. La híbrida no gana en una métrica única.

En el conjunto temporal y de alcance, las tres políticas acertaron las 3 ambiguas y el condicional fue el más rápido (5,8 s). La recomendación por puntuación de la evidencia

acertó 1 de 3 y no es un desambiguador fiable: el valor del condicional está en la cobertura, no en la recomendación, que se muestra solo como pista.

El conjunto v2 tiene 42 preguntas creadas por los autores; el silver asistido las etiquetó con un LLM y la doble anotación humana revisó la etiqueta de ambigüedad. Dejó 21 ambiguas y 21 de control (42/42, kappa 1,000). Cuatro preguntas que el silver asistido marcaba como ambiguas quedaron como control; dos tienen respuesta verificada (v-008 y v-009) y dos no (la tarifa del pasaje y el número de votos), así que la exactitud se mide sobre 40. El detector da F1 0,894 (precisión 0,81; recall 1,0); los cinco falsos positivos son esas cuatro y el proyecto Triángulo del Puma. El baseline cubre los tokens de referencia en 30 de 40 (0,750); la clarificación en 29 de 40 (0,725, p=1,0); el resultado de la política condicional es 33 de 40 (0,825), con cuatro pares a favor y uno en contra (McNemar p=0,375); la cobertura entre opciones es 18 de 21 (0,857). Sobre el silver asistido sube a 36 de 42 (p=0,039); la cobertura entre opciones es 22 de 25 (0,88). Los valores p comparan el resultado automático, no una evaluación semántica. Por tipo, acierta 1,0 en alcance, comparación, conversación y unidad, 0,78 en referencial y 0,75 en temporal. La recomendación se retira.

La lista plana tiene un límite. En la pregunta de los votos, el LLM propuso tres iniciativas y ninguna era la de 95 votos; la aumentación determinista añadió las cifras de la evidencia y la interpretación del silver entró como sexta opción. En el terremoto, la aumentación se contaminó con montos y unidades de otros documentos; el filtro por la unidad de la pregunta dejó las tres cifras de municipios y la de 472 entró como cuarta. Para preguntas con muchas interpretaciones, una clarificación jerárquica, que primero pregunte la faceta y después la opción, es el siguiente paso.

## XI. Limitaciones

1. **Texto sin cierre.** El 32,7 % de los documentos de la muestra no termina en un signo de cierre. Es un proxy de truncamiento, no una verificación de que el rastreador los cortó: una publicación completa puede acabar sin punto. La limpieza no puede recuperar ese texto y ningún recuperador puede encontrar lo que no está.
2. **Sesgo de fuentes.** Cuatro cabeceras concentran la prensa y Bluesky llega casi todo en inglés. El corpus no representa la conversación pública en español ni la diversidad de medios del país.
3. **Fechas incompletas.** El rango declarado va de 2008 a 2026, pero la distribución utilizable se concentra en 2024 a 2026. El filtro temporal solo es fiable en ese tramo.
4. **Silver pequeño y desbalanceado.** 43 preguntas, 5 sin respuesta y 24 en prueba; 36 son de prensa y 2 de Bluesky, frente a un corpus con 38,2 % de documentos de Bluesky. Los intervalos de confianza son amplios y las diferencias de una o dos preguntas no son concluyentes. La revisión de Erika está registrada; algunas correcciones las aplicó un solo revisor y las filas marcadas sin reemplazo siguen abiertas.
5. **Conjunto reservado pequeño.** 20 directas y 5 negativas. Una sola pregunta mueve 5 puntos porcentuales y los intervalos son amplios. La recuperación híbrida se eligió sobre este conjunto, así que sus métricas de recuperación y respuesta son de desarrollo; la compuerta se midió antes de elegir el híbrido, pero el mismo conjunto se usó después para esa elección, así que sus cifras también son de desarrollo.
6. **Recall con un solo documento.** El silver reservado tiene un documento de referencia por pregunta. Si otro documento contiene la misma respuesta, el recall lo cuenta como fallo; r-005, r-008, r-017 y r-019 responden bien sin el documento de referencia. El recall@10 de 0,700 es una cota inferior de la recuperación útil.
7. **Compuerta no concluyente.** La compuerta nueva rechaza las 5 negativas del conjunto reservado y 3 de 20 directas; dos de esos tres son preguntas que la recuperación híbrida sí puede responder, pero la compuerta solo ve el top-5 denso. La tasa real sigue sin estimarse con precisión.
8. **Ambigüedad con 15 preguntas únicas y 42 anotadas.** El v2 tiene 21 ambiguas y 21 de control. Las 15 son preguntas diseñadas por los autores; los conjuntos de 12 y 6 comparten tres controles. La doble anotación dio 42 de 42 etiquetas idénticas (kappa 1,000), pero el kappa mide la coincidencia en la etiqueta binaria de ambigüedad sobre 42 casos; no valida la corrección de las opciones ni la representatividad poblacional. Dos controles no tienen respuesta verificable en el corpus (la tarifa del pasaje y el número de votos) y se excluyen de la exactitud (n=40). El detector deja cinco falsos positivos y el condensador conserva el sujeto anterior en preguntas encadenadas, lo que hunde v-008. La conversación queda en 1 de 1 evaluable. La lista plana y la jerarquía siguen pendientes; la recomendación se retiró.
9. **Ruido del juez.** RAGAS depende de un modelo local que falla en algunas muestras y cambia con el timeout y la concurrencia. La comparación con la arquitectura anterior cambia juez y embedding, así que no es completamente equivalente. El artefacto mide la configuración densa con $k = 5$ y sin compuerta estricta, antes de incorporar el híbrido.
10. **Hardware e infraestructura.** La estación tiene 16 GB de VRAM y 31 GB de RAM. El contexto de 116K ocupa 13,9 GB; el reordenador en GPU añade 1,6 GB y el margen es estrecho. La capa de datos (etcd, MinIO y Milvus) corre como stack de Docker Swarm en un servidor de la red local y el API la alcanza por LAN en el puerto 19530, sin túnel. La aplicación con GPU sigue en Compose porque Swarm no pasa dispositivos al contenedor.
11. **Alcance.** El sistema está diseñado para un corpus concreto de prensa y redes. No es un RAG de propósito general y las decisiones de limpieza, chunking y compuerta no se transfieren sin medir.

## XII. Conclusiones y trabajo futuro

En el silver exploratorio el sistema responde 37 de 38 preguntas respondibles y se abstiene en las 5 sin respuesta. De las 37, 25 cubren los tokens de la respuesta de referencia; de las 12 restantes, la revisión manual de una persona juzgó correctas 8, parciales 3 e incorrecta 1. En el conjunto reservado entrega 19 de 20, cubre los tokens de referencia en 17 y se abstiene en las 5; con la compuerta activa entrega 15 de 20. En conversación alcanza 14 de 14 con compuerta y 11 de 14 sin ella.

La primera lección es metodológica: el piloto sobreestima la recuperación. Con el mismo codificador y las mismas preguntas, Recall@10 cae de 0,947 a 0,684 al pasar de 1.932 a 161.868 documentos. Una comparación de codificadores o de chunking hecha solo sobre una muestra pequeña no permite elegir con confianza. El conjunto reservado confirma la caída. La unión con BM25 y el reordenamiento recuperan parte de ella, pero no la cierran. La segunda es que el orden y la cobertura se atacan en sitios distintos. Reordenar el top-5 sube el nDCG 5,6 puntos y el P@1 13,1 sin perder Recall@10, pero no recupera lo que falta; la unión con BM25 sí recupera entidades que el denso no trae. La profundidad de la unión tampoco es monotónica: 50/50 recupera más documentos de referencia, pero baja la cobertura de tokens; 10/20 es el mejor equilibrio. La tercera es que la limpieza conservadora basta: ninguna variante supera a otra de forma significativa, y la adopción se apoya en la consistencia y en eliminar páginas de error, no en una mejora demostrada de la normalización.

La cuarta lección es que la abstención barata es la compuerta. Rechazar antes de generar cuesta decenas de milisegundos y evita que el modelo improvise sobre temas ajenos. La quinta es que la evaluación automática necesita cobertura: una media de RAGAS con 4 valores de 38 invirtió una conclusión sobre la precisión del contexto. Reportar $n$ junto a cada media no es un detalle de estilo; cambia lo que se concluye. La sexta lección es que, en estas muestras, el resultado automático del condicional es mayor que el del baseline: 33 de 40 frente a 30 de 40 en el silver humano, con cuatro pares a favor y uno en contra (McNemar p=0,375); la cobertura entre opciones es 18 de 21. La diferencia no alcanza significancia con 40 casos. En el silver asistido, 36 de 42 (p=0,039) y 22 de 25 de cobertura. La clarificación no se separa del baseline (29 de 40, p=1,0) y el detector baja a F1 0,894 por cinco falsos positivos. La recomendación por evidencia se retira. La clarificación interactiva da control pero cuesta un turno y una llamada. La política híbrida clarifica en la interfaz y condiciona en la API de una pasada. La séptima lección es que el silver exploratorio no basta. El conjunto reservado muestra el límite de la compuerta anterior. La compuerta con cross-encoder rechaza las 5 negativas y reduce los falsos rechazos, pero no los elimina; dos de los tres que quedan son preguntas que la recuperación híbrida sí puede responder. Un conjunto de 5 negativas no distingue una tasa nula de una tasa baja.

El trabajo futuro tiene seis frentes. Primero, cerrar los fallos de recuperación: g-046 exige revisar la consulta o el índice, y el conjunto reservado añade r-006 (abstención) y r-016 (pregunta ambigua). La recuperación híbrida ya resolvió r-012 y r-014. Segundo, alinear la compuerta con el conjunto reordenado, porque hoy rechaza dos preguntas que el generador podría responder. Tercero, mejorar la corrección de la respuesta, la métrica más débil de RAGAS. Cuarto, ampliar el conjunto de prueba para reducir los intervalos de confianza. Quinto, probar un chunking distinto y una reindexación completa, que es la única vía que queda para atacar la pérdida de contexto sin sacrificar cobertura. Sexto, probar una clarificación jerárquica para preguntas con muchas interpretaciones, porque la lista plana no las cubre todas.

## XIII. Reproducibilidad

### *XIII-A. Hardware y software*

El sistema se ejecutó en una estación con Ryzen 9 5950X, 31 GB de RAM y una GPU AMD Radeon RX 9070 XT de 16 GB con ROCm 7.2.4. La capa de datos (etcd, MinIO y Milvus 2.6.6) corre en un servidor de la red local con 16 núcleos y 62 GB de RAM; el API, el embedding y el generador siguen en la estación. El API se conecta a Milvus por LAN en `http://192.168.1.100:19530`; el stack de datos se despliega con Docker Swarm y la aplicación con Compose. Los contenedores usan Python 3.12, PyTorch 2.10 con ROCm, FastAPI, Streamlit y Ollama. RAGAS 0.4.3 y LangChain 0.3 corren en un contenedor aparte con Python 3.12 y `BAAI/bge-m3` en CPU.

### *XIII-B. Modelos*

La Tabla S6 del suplemento fija la versión de cada modelo. Los identificadores son las etiquetas de Ollama y de Hugging Face usadas en el código; no se usan alias móviles.

El generador, el detector y el condensador corren con temperatura 0; las repeticiones del mismo estado dan las mismas métricas. El artículo se compila con IEEEtran V1.8b y un parche local que sustituye la raya del resumen y de las palabras clave por dos puntos. La corrida de referencia de la configuración densa es el commit `125bbc7`; la de la configuración híbrida es `experimento_recuperacion_hibrida.py`. Las figuras y tablas se regeneran desde `reports/tablas/`. RAGAS 0.4.3 corre con LangChain 0.3 y `BAAI/bge-m3` en CPU.

### *XIII-C. Comandos*

El stack se levanta con un comando:

```
make up
make ps
curl http://localhost:8010/health
```

`make up` despliega la capa de datos en el servidor con `docker stack deploy -c docker-stack.yml taller-rag` y levanta la aplicación con `docker compose up -d -no-deps api frontend ollama`. El servicio `prewarm` carga el generador, el embedding y el reranker cuando la API está sana. La evaluación completa se ejecuta con:

```
python src/scripts/run_evaluation.py
```

Ese pipeline corre la evaluación de desempeño en el worker GPU, la de conversación contra la API y RAGAS en el contenedor de evaluación. Los resultados se escriben en `reports/tablas/` y la comparación en `reports/tablas/evaluacion_comparativa.md`.

### *XIII-D. Artefactos*

El silver consolidado está en `eval/silver/silver_standard.jsonl`, con su versión ampliada en `silver_standard_ampliado.jsonl`. El conjunto reservado (`silver_reservado.jsonl`, con hashes en `README_RESERVADO.md`) escribe `reports/corridas/reservado_*_raw.jsonl` y `reports/tablas/reservado_*_resumen.csv`. El silver de ambigüedad está en `eval/silver/silver_ambiguedad*.jsonl`, con las tablas `ambiguedad_*_resumen.csv` y `ambiguedad_*_por_pregunta.csv`. La capa de datos se define en `docker-stack.yml` y el modo local de respaldo en `docker-compose.data.yml`. Las figuras están en `reports/figuras/`, las corridas en `reports/corridas/` y los modelos de la compuerta en `modelos/`.

### *XIII-E. Datos*

El corpus completo no se publica por derechos de las fuentes. Se incluyen el silver, las anotaciones, la muestra del piloto y los Parquet de evaluación necesarios para reproducir las métricas. Las cifras de este artículo se pueden verificar contra esos artefactos sin volver a descargar el corpus.

## REFERENCIAS


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *NeurIPS Datasets and Benchmarks Track*, 2021, arXiv:2104.08663.

[3] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, arXiv:2210.07316.

[4] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, "MIRACL: A multilingual retrieval dataset covering 18 diverse languages," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1114–1131, 2023, arXiv:2210.09984.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[6] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, arXiv:2311.09476.

[7] A. Salemi and H. Zamani, "Evaluating retrieval quality in retrieval-augmented generation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, arXiv:2404.13781.

[8] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, P. Li *et al.*, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2408.08067.

[9] Zilliz, "Milvus documentation," https://milvus.io/docs, 2026.

[10] Ollama, "Ollama: Get up and running with large language models locally," https://github.com/ollama/ollama, 2026.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference*, 2009, pp. 758–759.

[13] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[14] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[15] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[16] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[17] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[18] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.

[20] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," *arXiv preprint arXiv:2401.00396*, 2024.

[21] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com, 2026.

[22] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io, 2026.

[23] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 5783–5797.

[24] L. Kuhn, Y. Gal, and S. Farquhar, "CLAM: Selective clarification for ambiguous questions with generative language models," *arXiv preprint arXiv:2212.07769*, 2022.

[25] T. Zhang, P. Qin, Y. Deng, C. Huang, W. Lei, J. Liu, D. Jin, H. Liang, and T.-S. Chua, "CLAMBER: A benchmark of identifying and clarifying ambiguous information needs in large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 10 746–10 766.

[26] M. J. Zhang and E. Choi, "Clarify when necessary: Resolving ambiguity through interaction with lms," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 5541–5558.

[27] R. Sahay, L. S. Tekumalla, P. Aggarwal, A. Jain, and A. Saladi, "ASK: Aspects and retrieval based hybrid clarification in task oriented dialogue systems," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Industry Track*, 2025, pp. 881–895.

[28] G. Kim, S. Kim, B. Jeon, J. Park, and J. Kang, "Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[29] Y. In, S. Kim, R. A. Rossi, M. M. Tanjim, T. Yu, R. Sinha, and C. Park, "Diversify-verify-adapt: Efficient and robust retrieval-augmented ambiguous question answering," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 1212–1233.

[30] Z. Li, Y. Li, H. Xie, and S. J. Qin, "CondAmbigQA: A benchmark and dataset for conditional ambiguous question answering," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

[31] M. Tanjim, Y. In, X. Chen, V. Bursztyn, R. A. Rossi, S. Kim, G.-J. Ren, V. Muppala, S. Jiang, Y. Kim, and C. Park, "Disambiguation in conversational question answering in the era of llms and agents: A survey," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 9537–9550.